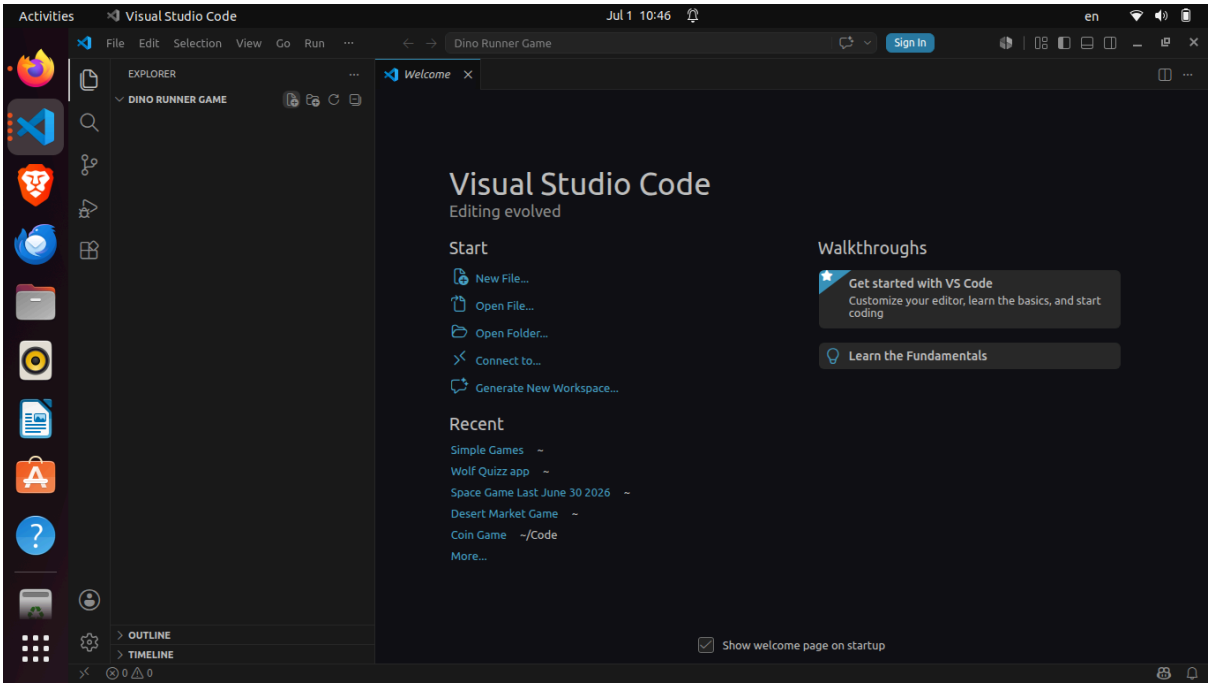
Dino Runner Game - Kid Coder Guide

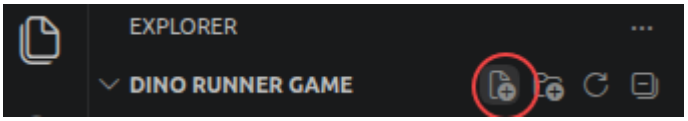
Step by Step - all resources and explanations provided

Step 1

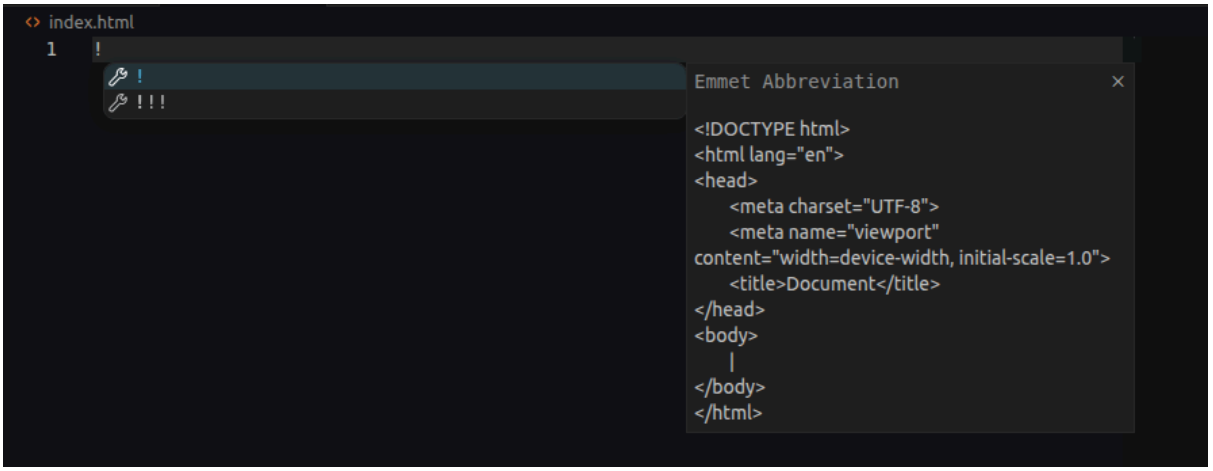
As always, open VS Code, click on “File” at the top bar, choose “open folder”, find a suitable folder for your code project eg a folder called Code Projects, then create a new folder we can call Dino Runner, select this folder and open in VS Code.



In the left sidebar, click on the new file icon that appears next to your folder name. Call it index.html (or dino.html if you prefer!)



Then, inside your HTML file simply type one letter or to be precise, a single character: ! and press enter, which will auto-suggest the full basic HTML page setup, or template.



We start with a basic HTML template (a template is basically pre-typed code with what you need then you can add or change what you want. It saves time.)

You should now have:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width,
initial-scale=1.0">
   <title>Document</title>
</head>
<body>

</body>
</html>
```

Change the title from Document to Dino Runner, or whatever you want to call it. This will appear in the tab in your browser.

```
   <title>Dino Runner</title>
```

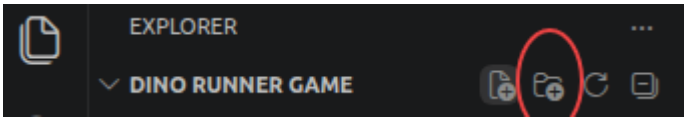
Step 2

Download your resources or assets

In our game we only have 3 image files to download.

First create a new folder inside our Dino Game.

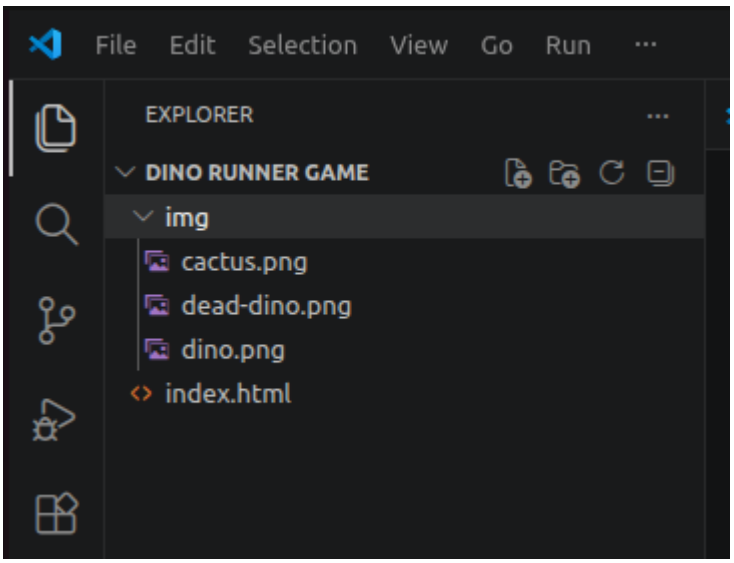
Click on the New Folder icon:



Call it img - it's just an easy name for your images folder. You can call it "images" if you prefer.

Now download the 3 images attached to this worksheet, and move them to your img folder.

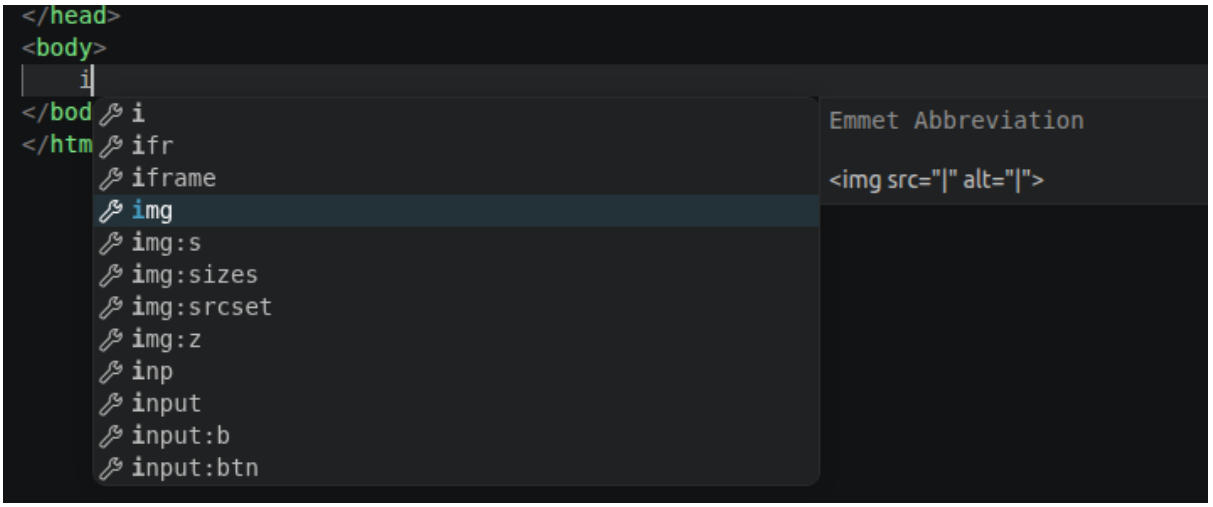
You should now see this in your sidebar:



If you have any trouble handling your files, moving them to the right spot, or anything else, please contact me and I'll try and guide you.

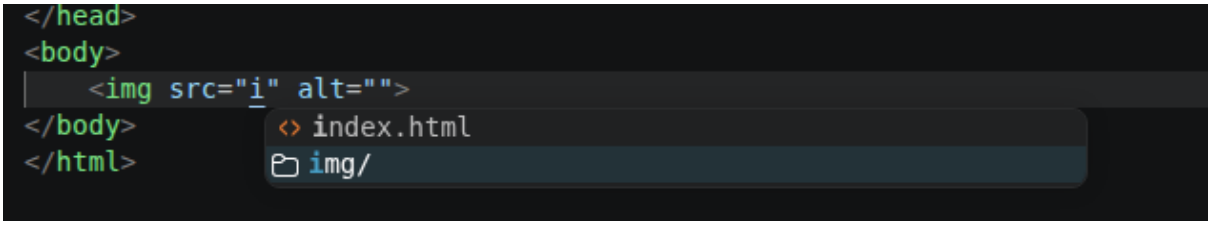
Alternatively, you can use a link to these images stored on my website.

Now, back to the code. Inside the body, simply type i. VS Code will suggest a menu. Choose img, like this. You can use the down arrow to reach img then press enter, or just click on it.

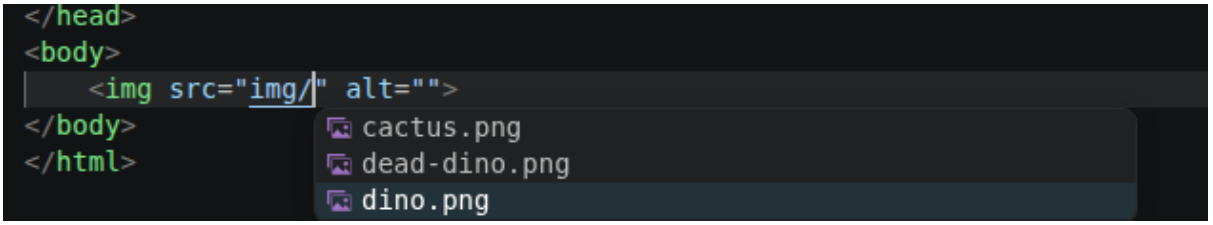



```
<body>
    <img src="" alt="">
</body>
```

We now need to provide a source for our image. Inside the src="" type i again.



Choose your img folder…



…and then your dino image.

Follow this process for each image:

```
<body>
   <img src="img/dino.png" alt="dino">
   <img src="img/cactus.png" alt="cactus">
   <img src="img/dead-dino.png" alt="dead dino">
</body>
```

Remember to fill out the alt or alternative text in case the image does not appear - eg the file does not exist.

Great! We have our images, lets see if everything is working well.
Right-click on the index.html file name, select the copy path option from the menu, and paste into your browser. You should see some large images filling your screen. Scroll down to see all 3.

Step 3
Styling

We need to style each image differently.
Comment
First, comment out the dead dino image - we don't want to see him until game over. We just want to make sure it’s working.
To make a comment or to “comment out” some code so it won’t be read by the browser, simply click on the line you want to comment and press Control and / (same button as the ?). On apple or mac computers its command and /.
Or you can just type <!-- before your comment and - -> at the end.

ID
Give the cactus and dino an id:

```
<body>
   <img id="dino" src="img/dino.png" alt="dino">
   <img id="cactus" src="img/cactus.png" alt="cactus">
   <!-- <img src="img/dead-dino.png" alt="dead dino"> -->
</body>
```

Now, inside the head, right after the title, type “st” for style.
Press enter.

Body

We start by styling the body - the entire page.

```
        body{
            background-color: skyblue;
            margin: 0;
        }
```

Margin 0 means that there will be no margin between the edge of the screen and your game images. This makes it easier to control, and to know where everything is, when your images start at 0 instead of a few pixels in.
Check your page in the browser, press refresh. Look for ⟳ at the top.
Is your screen sky blue?

Images style

```
        img{
            height: 100px;
            position: absolute;
            bottom: 100px;
        }
```

Explanations:

Height changes to just 100px high instead of taking up the whole page, no need to change width - with images that changes automatically. If you do change the width too, that will often stretch your image. So generally, we only change the width or the height - whichever we care about more. In our case, we want both images to be the same height, so the dino can jump 100px and clear the cactus. If we’d only define the width, we may end up with a cactus higher than 100px.

Position absolute allows us to move each image exactly where we want on the screen using top, bottom, left, right.
Bottom 100px places them above the ground we will add soon which will be 100px high.

If you want to move the images wherever you want, have fun with the next two CSS rules. (Each CSS word and {} is called a rule.)

```
#dino{
    left: 0;
}
#cactus{
    left: 500px;
}
```

You can add top, right, bottom, left properties (each CSS keyword used before the ":" is called a property, what's after, usually a number, is called a value) and see your images placed in various positions around the screen. Have fun getting used to controlling the image positions!

For example:

```
#dino{
    right: 245px;
    top: 345px;
}
#cactus{
    left: 500px;
    bottom: 456px;
}
```

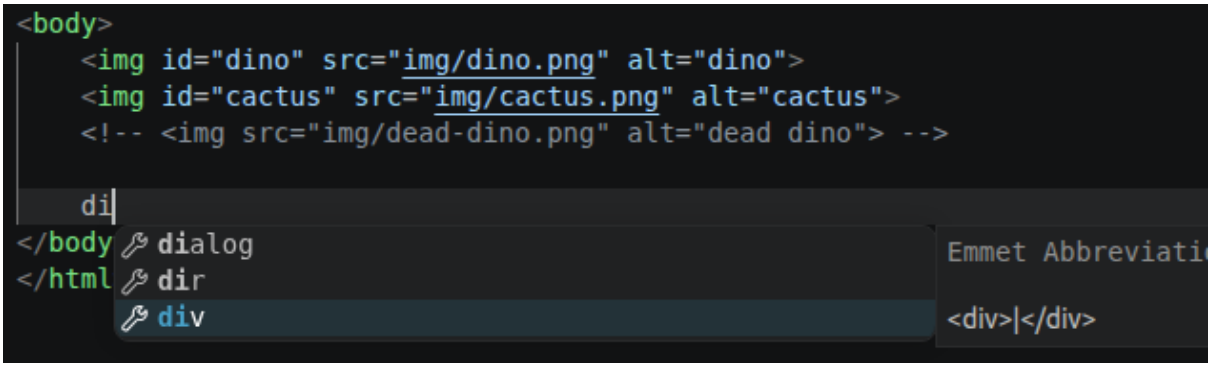
But for now this is good:

```
#dino{
    left: 0;
}
#cactus{
    left: 500px;
}
```

We don't actually need to say left: 0, because by default, whenever you say position absolute, the position will be top: 0; left: 0. Note we don't need to add px since 0cm is the same as 0px. But any other value requires specifying that it's 500 pixels and not 500 cm!

Ground

We will use a new HTML element for the ground - a div:



Div stands for division, and is usually used to divide the screen into sections.
Often we add more elements inside the div, for example, a card may have an image, heading and paragraph.
Don't forget to give it an ID!

```
<div id="ground">Ground</div>
```

We type "Ground" just so we can see it's there. Later we will delete the word "Ground".
Check your browser to make sure it's there, no spelling errors or bugs. In general it's worth checking your browser and refreshing ⟳ every time you change something to see if everything is working well.

Now let's style it:

```
#ground{
    background-color: tan;
    width: 100%;
    height: 100px;
    position: absolute;
    bottom: 0;
}
```

The styling should be quite easy to understand.
We want the ground to take up the entire width of the page so we use width 100%.
Position absolute allows us to move it to the bottom using bottom: 0.

I now see the dino standing safely on the ground instead of midair - but the cactus is a few mm above the ground:

Let’s play around with the bottom property of our cactus until we get the right position.
Let’s try bottom: 95px;

This is close but I notice that my cactus slips behind the ground!
This is because the ground element is coded after the cactus element.
So first the browser reads the code, line by line, it first draws the cactus and then a tiny millisecond later it draws the ground on top of the cactus!
We can solve this by placing the ground above the cactus in the html like this:

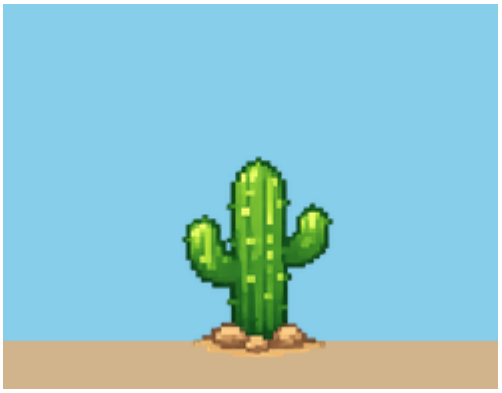
```
<body>
    <div id="ground">Ground</div>

    <img id="dino" src="img/dino.png" alt="dino">
    <img id="cactus" src="img/cactus.png" alt="cactus">
    <!-- <img src="img/dead-dino.png" alt="dead dino"> -->
```

- but this solution won’t always work. Sometimes it’s not practical to organise all your code in order of which element goes in front of which.
CSS has another solution: z-index.
To put it simply, z-index decides who goes in front - the higher your number you go on top.
So inside the img rule let’s add 1 line: `z-index: 1;`

```
img{
    height: 100px;
    position: absolute;
    bottom: 100px;
    z-index: 1;
}
```

Success!
My cactus is now in front of the ground:

I keep playing around with the cactus bottom property, and I find that bottom: 92px looks best. Of course you can choose which exact position you believe looks best for your style.



Step 4
Jump button
In your body add:

```
    <button onclick="jump()">Jump</button>
```

Make sure the onclick is inside the first button tag.
Now add a script tag.
Just type “sc” and press enter.
This should go at the end of your body, after all your html elements.
Like this:

```
<body>
    <img id="dino" src="img/dino.png" alt="dino">
    <img id="cactus" src="img/cactus.png" alt="cactus">
    <!-- <img src="img/dead-dino.png" alt="dead dino"> →

    <div id="ground"></div>

    <button onclick="jump()">Jump</button>
```

```
    <script>

    </script>
</body>
```

Now let's create the jump function.
It's very simple for now, we just want the dino to move up 100px.
Right now, bottom is 100px, just above the ground, so let's change bottom to 200px:

```
    <button onclick="jump()">Jump</button>
    <script>
        function jump(){
            document.getElementById("dino").style.bottom = "200px"
        }
    </script>
```

Now when you click on the button, dino jumps up!
Unfortunately we have not made a way for him to get down again so he's left hanging in the air!
Let's make another button to get him down again, and another function. It's easiest to just copy and paste (Control C to copy, Control V to paste) the button and the function and just change a few words:

```
    <button onclick="jump()">Jump</button>
    <button onclick="jumpDown()">Jump Down</button>
    <script>
        function jump(){
            document.getElementById("dino").style.bottom = "200px"
        }
        function jumpDown(){
            document.getElementById("dino").style.bottom = "100px"
        }
    </script>
```

Remember that JavaScript is case sensitive meaning there's a difference between upper case "D" and lower case "d".
(So onclick="jumpdown()" …. function jumpDown() will not work)

How do we make the dino jump down automatically after half a second?

Step 5
Set Timeout
Set timeout in JS basically waits a set amount of time, then runs a function, essentially "pressing the button" for you.
The time is measured in milliseconds, so to wait one second you just put 1000 there.

We want to wait half a second.

```
    <script>
        function jump(){
            document.getElementById("dino").style.bottom = "200px"
            setTimeout(jumpDown, 500)
        }
        function jumpDown(){
            document.getElementById("dino").style.bottom = "100px"
        }
    </script>
```

Check it out in the browser. The dino should jump up when you press the “Jump” button, wait half a second then jump down again.
We don’t need the Jump Down button anymore. Let's delete it. Or - change it for the next step.

Step 6
Make the cactus move.
First lets make the cactus move manually using a button, then we’ll automate it.
In the html add:

```
    <button onclick="moveCactus()">Move Cactus</button>
```

In the script we will need a variable for the cactus left position, since that's going to change each time we press the button.
Here's the code for the button and function:

```
    <button onclick="moveCactus()">Move Cactus</button>
    <script>
        let cactusLeft = 500
        function moveCactus(){
            cactusLeft -= 5
            document.getElementById("cactus").style.left = cactusLeft +
"px"
        }
```

Cactus left -= 5 means we subtract 5 from cactus left each time the function is triggered by pressing the button.
Try in the browser - does the cactus move each time you click on Move Cactus?

Step 7
Set Interval
Set Interval is very similar to setTimeout, but this runs a function constantly, only waiting an interval of time between each time it “presses the button” for you.
We add just one new line of code after our function: `setInterval(moveCactus, 20)`

```
        let cactusLeft = 500
        function moveCactus(){
            cactusLeft -= 5
            document.getElementById("cactus").style.left = cactusLeft +
"px"
        }
        setInterval(moveCactus, 20)
```

Step 8

Check collision

When the cactus reaches or “collides” with the left edge of the screen it disappears because the left position becomes negative. So obviously, you can’t see it if the screen begins at 0 pixels on the left and the cactus' left position is now -648 pixels!

How do we know if the cactus has left the screen?

Very simple, inside our moveCactus function, after changing the cactusLeft value, we check:

```
if(cactusLeft < 0)
```

What’s next?

What do we want to happen when the cactus moves off the left edge

We want it to jump to the right edge.

Now every screen will be different, it depends on the size of your computer screen. Some screens will be 1000 pixels wide, others will be more or less.

So we use window.innerWidth to find the width of your screen.

So the full moveCactus function is now:

```
    <button onclick="moveCactus()">Move Cactus</button>
    <script>
        let cactusLeft = 500
        function moveCactus(){
            cactusLeft -= 5
            if(cactusLeft < 0){
                cactusLeft = window.innerWidth
            }
            document.getElementById("cactus").style.left = cactusLeft +
"px"
        }
        setInterval(moveCactus, 20)
```

We can now improve our code.

First let’s delete the Move Cactus button. No need for it anymore.

Second, let's make the cactus begin on the extreme right side of the screen.

So lets change the catusLeft variable to:

```
        let cactusLeft = window.innerWidth
```

Step 9
Add a Start button
We can simply move our setInterval inside a function and trigger it using a button:

```
    <button onclick="start()">Start</button>
    <script>
        let cactusLeft = window.innerWidth
        function start(){
            setInterval(moveCactus, 20)
        }
```

Try it in the browser.
What happens when you press start a few times?
That's right, it speeds up.
Why?

Let's think of it like this. Each time you press the button, you create a robot that will trigger the moveCactus function, as if it's pressing the old Move Cactus button. If we have just one "robot" activating the function we will have the cactus move 5 pixels every 20 ms, or 50 times a second (1000/20=50). When you press the start button again you "create" another robot, also moving the cactus 5 pixels 50x a second. So now it doubles the speed, 10px each time. So 10 clicks on the Start button will make it move 50 pixels 50x a second or the full width of the screen in less than half a second. You can imagine ten robots pressing the (now deleted) Move Cactus button 50 times a second as opposed to what we want, just 1!

We need to find a way to prevent the start function from running setInterval after 1 click. There are a few ways to do this. See if you can think of other ways.
But we will use the standard method.

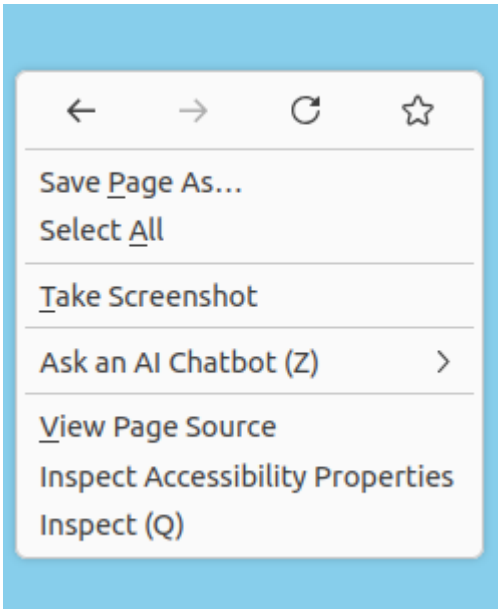
Each time setInterval is activated, so to speak, creating another robot, each "robot" is given an ID, usually simply 1, 2, 3, or something like that. You can check this yourself in the console.

```
        let intervalID = 0
        function start(){
            intervalID = setInterval(moveCactus, 20)
            console.log("Robot Number: ", intervalID, " has just been
created!")
        }
```

Now setInterval both activates moveCactus AND provides us with an interval ID.
We can check the console to see what ID number it gave the "robot" each time a new one is created.

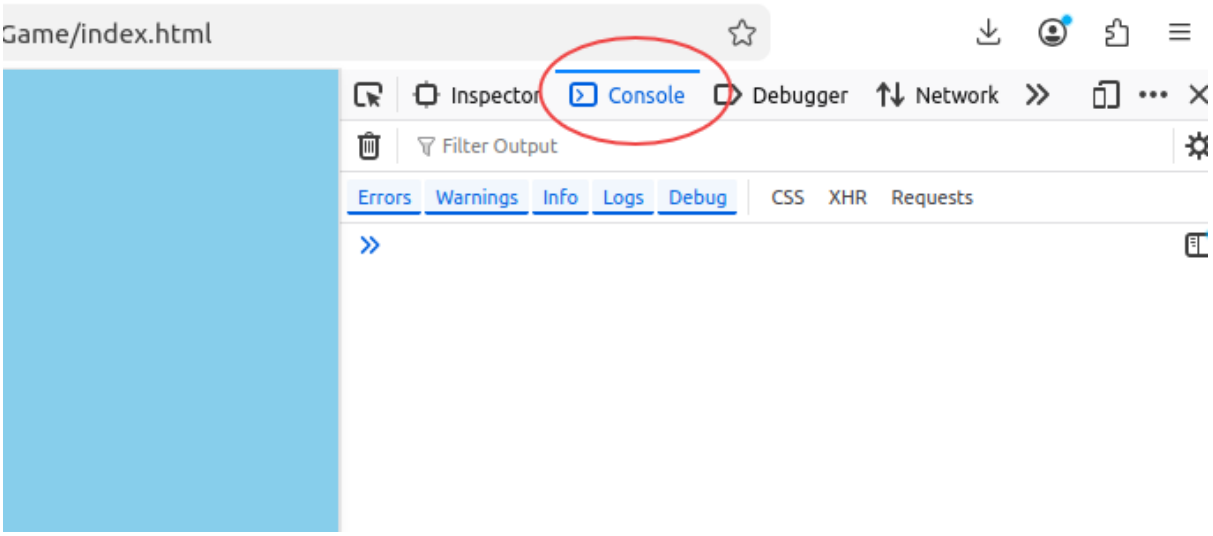
Using the Browser Console

To open your console, simply right-click with your mouse anywhere on the page in your browser. You should get a menu that looks a bit like this:

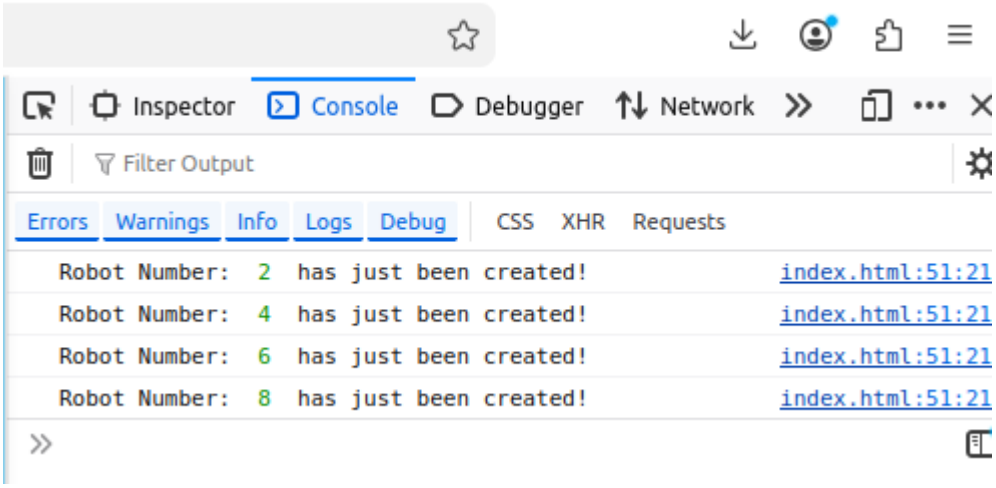


Select Inspect (Q), opening the Developer Tools sidebar. If you right-click while pressing the Q key, the Dev Tools will open automatically.
Alternatively, press F12 to open Dev Tools. You can find it on the top row of your keyboard:

Inside Dev Tools, click on the console tab:



We want to watch our “robots” being created.
Click on the Start button a few times.
You should see something like this:

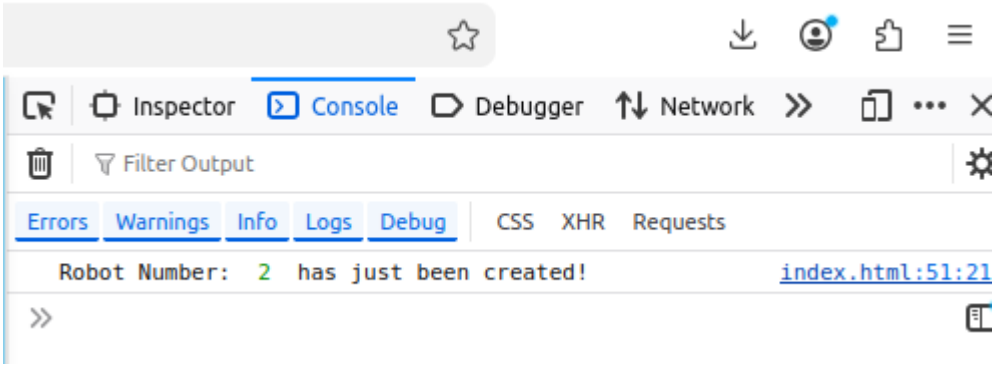


My browser decided to give each robot IDs of 2, 4, 6, 8 etc.
Sometimes it’s a different pattern.
It doesn’t matter, as long as we can track each robot and control it using its ID.

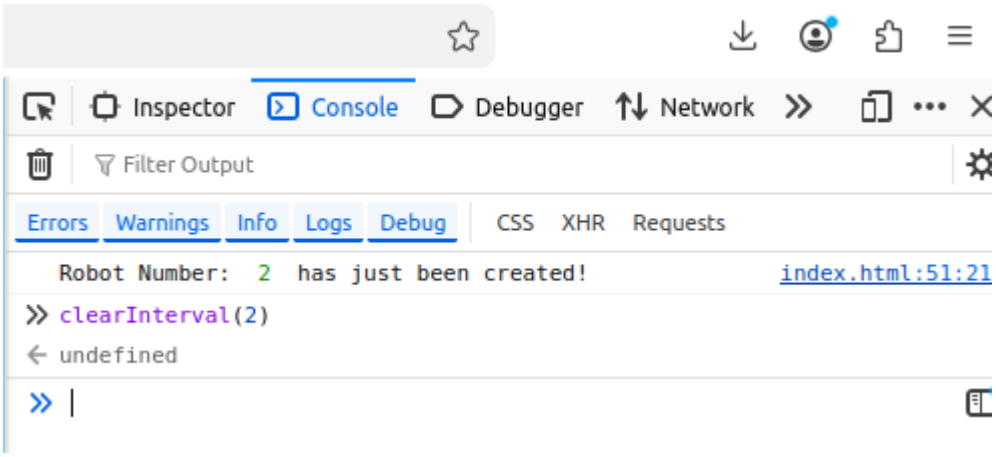
Step 10
Adding a Stop button

How do we stop a setInterval from running?
First we need to know which “robot” is activating the function “clicking the button”, call it by ID and tell it to stop. We can do this very simply now in the console.

Refresh ⟳ the page and press the Start button just once.
You should see something like this:

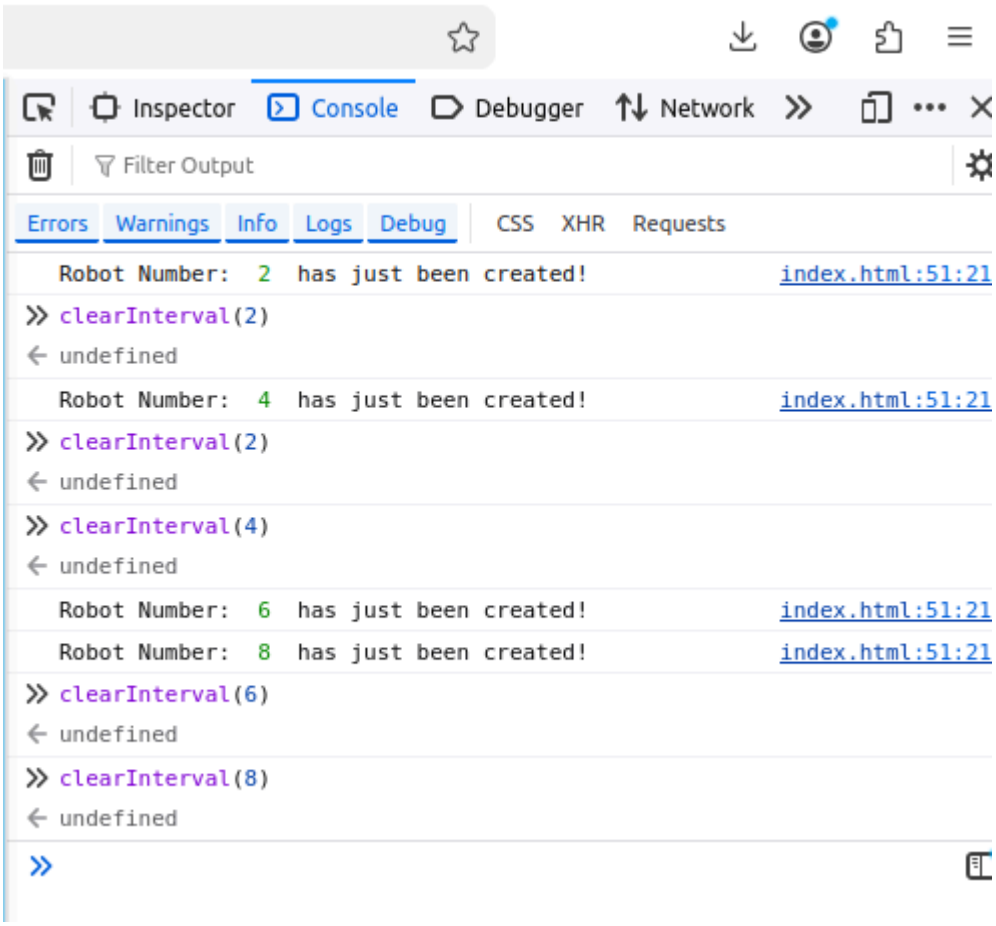


Now click your mouse by the >> and type clearInterval(2) - or whatever is the ID for your robot. Usually you can just type “cl” and a menu will appear suggesting clearInterval among other options.
Press Enter.



The cactus should stop moving.

You can play around.
To repeat a line that was used before just press the up arrow key, and the last line entered into the console will appear. This helps save time.
Start and stop the cactus, press start a few times. You will see that you must stop each robot using its ID.
Here’s a log of my playing around.



Brief explanation of what my console is showing:

(I press Start once.)
Robot Number: 2 has just been created! index.html:51:21
clearInterval(2)
undefined
— (Cactus stops) —

(I press start again)
Robot Number: 4 has just been created! index.html:51:21
clearInterval(2)
undefined
— (Cactus does NOT stop, its new ID is 4 and I used 2) —

clearInterval(4)
undefined
— (Cactus stops) —

(I press Start twice)
Robot Number: 6 has just been created! index.html:51:21
Robot Number: 8 has just been created! Index.html:51:21

— (Cactus speeds up, double speed - two “robots”) —

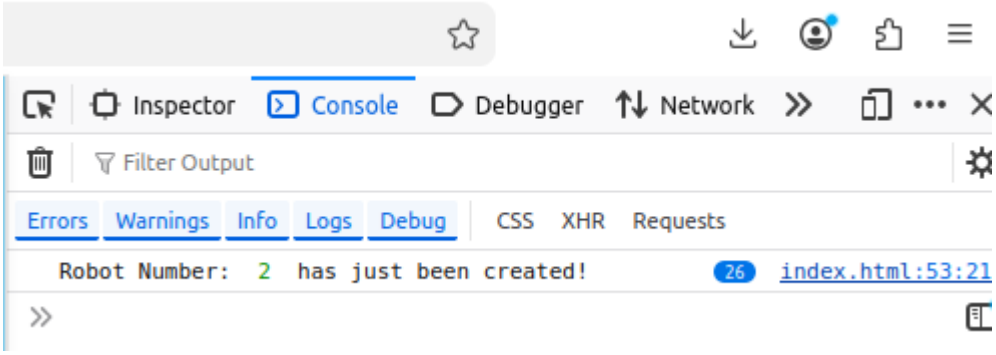
clearInterval(6)
undefined
— (First robot stops, cactus slows down) —

clearInterval(8)
undefined
— (Cactus stops) —

First, let's prevent the Start button from activating more than once. We simply only set the interval IF interval ID is 0. If it’s not 0, it must be we created a robot, so no more setIntervals allowed:

```
        let intervalID = 0
        function start(){
            if(intervalID == 0){
                intervalID = setInterval(moveCactus, 20)
            }
            console.log("Robot Number: ", intervalID, " has just been
created!")

        }
```

The result in the console - I can press Start 26 times but our robot keeps the same ID and no new robots are created:



Lets add a Stop button:

```
    <button onclick="start()">Start</button>
    <button onclick="stopGame()">Stop</button>
    <script>
```

And in the script:

```
        function stopGame(){
            clearInterval(intervalID)
            intervalID = 0 // Reset, so Start can work again
        }
```

When we click Stop, clearInterval(intervalID) stops the robot.
But intervalID still holds the old robot number.

So we set intervalID back to 0.

This means: “No robot is running now.”

Then, when we click Start again, if(intervalID == 0) is true, so the program allows a new robot to start.

Without this, Start would not work again, because the program would still think a robot is already running.

Step 11

Increase Score

There are many ways to score points in a game.

We will use the Dino Runner system - very simply, you score a point each 20ms your dino is still alive. Why 20ms? Because that's the easiest, since our interval runs our moveCactus function every 20ms - or 50x a second.

In our HTML section, right after body, let’s add a div or p tag.

```
<body>

   <p id="score">Score: 0</p>
```

In our script, we can first create a score variable, then increase it inside the moveCactus function, then update our HTML score element with new text.

Heres our moveCactus function now with a new score variable created, and 2 lines added to the function:

```
let score = 0
function moveCactus(){
    score++ // add 1 to score
    document.getElementById("score").innerText = "Score: " + score

    cactusLeft -= 5
    if(cactusLeft < 0){
        cactusLeft = window.innerWidth
    }
    document.getElementById("cactus").style.left = cactusLeft + "px"
}
```

Step 12
Refactor Our Functions

This is a very important coding skill.

Refactor means: clean up the code without changing what the code does.

At first, refactoring may feel unnecessary. After all, the game already works!

But refactoring helps us think more clearly. It makes the code easier to read, easier to understand, and easier to fix when something goes wrong.

Our moveCactus function is starting to do a few different jobs:

It moves the cactus left.
It checks if the cactus left the screen.
It sends the cactus back to the right side.

And now we just added the score update - which is really very different from the job of the function: to move the cactus.

So now we will split one bigger function into a few smaller functions.

The game will work the same, but our code will be cleaner and easier to improve.

First, since we want to change two things with setInterval, let's create a game loop. A loop is just a piece of code that is run again and again, many times over like running in a loop.

So inside our start function lets change it from:

```
        function start(){
            if(intervalID == 0){
                intervalID = setInterval(moveCactus, 20)
            }
            console.log("Robot Number: ", intervalID, " has just been
created!")
        }
```

To:

```
        function start(){
            if(intervalID == 0){
                intervalID = setInterval(gameLoop, 20)
            }
        }
```

We just changed the function from moveCactus to gameLoop, a function which we will create now.

```
        function gameLoop(){
            moveCactus()
        }
```

Now lets simplify the move cactus function.
Let's start with the score code since that's clearly a separate function and now belongs in the game loop function since it needs to be activated 50x a second.

```
    function addToScore(){
        score++ // add 1 to score
        document.getElementById("score").innerText = "Score: " + score
    }
```

The move cactus code goes from:

```
let score = 0
function moveCactus(){
    score++ // add 1 to score
    document.getElementById("score").innerText = "Score: " + score

    cactusLeft -= 5
    if(cactusLeft < 0){
        cactusLeft = window.innerWidth
    }
    document.getElementById("cactus").style.left = cactusLeft + "px"
}
```

To:

```
let score = 0
function moveCactus(){
    addToScore()
    cactusLeft -= 5
    if(cactusLeft < 0){
        cactusLeft = window.innerWidth
    }
    document.getElementById("cactus").style.left = cactusLeft + "px"
}
function addToScore(){
    score++ // add 1 to score
    document.getElementById("score").innerText = "Score: " + score
}
```

We now move the add to score function to the game loop:

```
    function gameLoop(){
        moveCactus()
        addToScore()
    }
```

Make sure to cut or delete the add to score function from the move cactus function, and paste it in the game loop.

Now let's create another function, to check if the cactus has reached the edge of the screen. Again, this is really a separate job, logically speaking. The move cactus function just moves the cactus. We should separate the job to check if the cactus has left the screen.
So we take these few lines of code:

```
    if(cactusLeft < 0){
        cactusLeft = window.innerWidth
    }
```

And place it in its own function:

```
function checkCactusHitEdge(){
    if(cactusLeft < 0){
        cactusLeft = window.innerWidth
    }
}
```

Now our move cactus function is already looking a lot simpler and easier to read:

```
let score = 0
function moveCactus(){
    cactusLeft -= 5
    checkCactusHitEdge()
    document.getElementById("cactus").style.left = cactusLeft + "px"
}
```

Step 13
Check if Cactus has Crashed into Dino
(Actually, since our poor Dino is supposed to be running, technically from the game play perspective, we are checking if the dino has crashed into the cactus!)

We're almost there!
The only main game mechanic left to create is making game over when cactus touches dino.

Let's split that into 2 functions.
Check if the cactus crashed into the dino, and the game over state.
First, how can we know if the cactus is touching the dino?
Well, we know from our styling that the dino image is 100 pixels wide, starting from 0 pixels ie the left edge of the screen.
So if the cactus left side is less than 100px, then we may be touching the dino…if, and only if, dino is not jumping.

So we really need to check just two things:
Is the cactus' left side less than 100?
Is the dino jumping?

Checking cactus less than 100 is very easy:

```
function checkCrash(){
    if(cactusLeft < 100){
        gameOver()
    }
}

function gameLoop(){
    moveCactus()
    addToScore()
    checkCrash()
}
```

We just added one more function to the game loop, the check crash function, and all it does for now is call the game over function (which we will create in a minute) if cactusLeft < 100.

Can you think of a way to check if the dino is jumping?
Where do we make him jump?
You click on the Jump button and call the jump function.
So let's add a variable inside the jump function that changes when you call it.
First lets create a variable called dino is jumping:

```
let dinoIsJumping = false
```

This is a new type of variable called a boolean. It can be either true or false.
We could have used 1 or 0 instead, but true and false are clearer and sound like real English.
Here's our new jump and jump down functions:

```
let dinoIsJumping = false
function jump(){
    dinoIsJumping = true
    document.getElementById("dino").style.bottom = "200px"
    setTimeout(jumpDown, 500)
}
function jumpDown(){
    dinoIsJumping = false
    document.getElementById("dino").style.bottom = "100px"
}
```

And lets add this to our check crash function:

```
function checkCrash(){
    if(cactusLeft < 100 && dinoIsJumping == false){
        gameOver()
    }
}
```

Great!
Now all we need to do is decide what to do with game over!
For now, just 2 things.
Stop the game loop - ie stop the cactus from moving and stop the score going up.
And turn the dino image into a dead dino. For now that's really enough.

Here's our game over function:

```
function gameOver(){
    stopGame()
    document.getElementById("dino").src = "img/dead-dino.png"
}
```

I copied the new src file path from the commented out image of the dead dino. To be honest, that's a big reason why I made it. It's easier to use the vs code app autosuggest tool to find the file when creating the HTML element. It doesn't always work so simply in js.

That's it!
Our game is practically done, now let's just test it.

I just played the game, and one thing I noticed, it's very hard to jump without the dino dying. That can be solved simply by making our set timeout function wait a bit longer ie a longer time out.
Find this line in the jump function and edit it to 600, 650 or 700, whichever you prefer:

```
        setTimeout(jumpDown, 700)
```

Play around to find which number works best, not too easy, not too hard.

Another thing I noticed is that the dead dino is hovering in the air. If you like that idea - maybe it's the soul hovering! Then fine. But I think I might prefer some realism for this game. Can you work out how to solve this?
It's really quite simple, you should be able to do it using what we've learned so far.
Solution on the next page…

Inside the game over function add one line to lower the dino from its original 100px:

```
        function gameOver(){
            stopGame()
            document.getElementById("dino").style.bottom = "80px"
            document.getElementById("dino").src = "img/dead-dino.png"
        }
```

Well done!
We have a complete game!
We can still add a lot of improvements like high score, a button to restart the game after losing instead of having to press refresh ⟳ each time etc but this is a great first game.
Later we can also style the buttons, add a nice display screen, maybe some scenery…
What do you think?
What else would you like to see in the game?

Here's the full code to check in case you made any errors, or something doesn't seem to work right:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width,
initial-scale=1.0">
   <title>Dino Runner</title>
   <style>
       body{
           background-color: skyblue;
           margin: 0;
       }

       img{
           height: 100px;
           position: absolute;
           bottom: 100px;
           z-index: 1;
       }
       #dino{
           left: 0;
       }
```

```
        #cactus{
            left: 500px;
            bottom: 92px;
        }
        #ground{
            background-color: tan;
            width: 100%;
            height: 100px;
            position: absolute;
            bottom: 0;
        }
    </style>
</head>

<body>
    <p id="score">Score: 0</p>

    <img id="dino" src="img/dino.png" alt="dino">
    <img id="cactus" src="img/cactus.png" alt="cactus">
    <!-- <img src="img/dead-dino.png" alt="dead dino"> -->

    <div id="ground"></div>

    <button onclick="jump()">Jump</button>

    <button onclick="start()">Start</button>
    <button onclick="stopGame()">Stop</button>
    <script>
    let cactusLeft = window.innerWidth

    let intervalID = 0

    function start(){
        if(intervalID == 0){
            intervalID = setInterval(gameLoop, 20)
        }
    }

    function stopGame(){
        clearInterval(intervalID)
        intervalID = 0 // Reset, so Start can work again
    }
```

```
let score = 0
function moveCactus(){
    addToScore()
    cactusLeft -= 5
    checkCactusHitEdge()
    document.getElementById("cactus").style.left = cactusLeft + "px"
}
function addToScore(){
    score++ // add 1 to score
    document.getElementById("score").innerText = "Score: " + score
}

function checkCactusHitEdge(){
    if(cactusLeft < 0){
        cactusLeft = window.innerWidth
    }
}

let dinoIsJumping = false

function jump(){
    dinoIsJumping = true
    document.getElementById("dino").style.bottom = "200px"
    setTimeout(jumpDown, 700)
}
function jumpDown(){
    dinoIsJumping = false
    document.getElementById("dino").style.bottom = "100px"
}

function checkCrash(){
    if(cactusLeft < 100 && dinoIsJumping == false){
        gameOver()
    }
}

function gameOver(){
    stopGame()
    document.getElementById("dino").style.bottom = "80px"
    document.getElementById("dino").src = "img/dead-dino.png"
}
```

```
    function gameLoop(){
        moveCactus()
        addToScore()
        checkCrash()
    }
    </script>
</body>
</html>
```